

KubeQuest — Jeu de commandes pour la démo (annoté en français)

Usage : la plupart des commandes se lancent après `ssh node1` (kubectl depuis le control-plane). Celles marquées « poste » se lancent sur votre PC. À préparer : ① les 4 terminaux nœuds ouverts ; ② Grafana / ArgoCD déjà ouverts dans le navigateur ; ③ ce document sous les yeux. Format de chaque étape : **【Dire】** ce qu'on dit au jury → **【Lancer】** la commande → **【Voir】** le résultat attendu.

0. Ouverture : connexion & vue d'ensemble

【Dire】 « Voici notre cluster Kubernetes de 4 nœuds construit avec kubeadm, à rôles séparés : control-plane, worker, ingress, monitoring. » **【Lancer】** (node1)

```
kubectl get nodes -o wide
```

【Voir】 Les 4 nœuds `Ready`, v1.31.14, avec leurs IP internes.

【Dire】 « Le cluster est sain, aucun pod anormal. » **【Lancer】**

```
kubectl get pods -A | grep -vE "Running|Completed" ; echo "=> vide au-dessus = tout est sain"
```

【Voir】 Seule la ligne d'info s'affiche = rien d'anormal.

1. Application (convertie de docker-compose vers Helm)

【Dire】 « À l'origine en docker-compose, l'application est packagée en Helm chart, 2 réplicas, avec MySQL persistant. » **【Lancer】**

```
kubectl -n laravel-app-prod get pods,svc
kubectl -n laravel get statefulset mysql
```

【Voir】 `laravel-app` 2/2 Running ; StatefulSet MySQL 1/1.

【Dire】 « L'application est accessible depuis l'extérieur. » **【Lancer】** (navigateur, poste) ouvrir : `http://larapp.35.156.165.130.nip.io` **【Voir】** La page compteur ; le bouton +1 incrémente.

2. Haute dispo + auto-scaling (HPA) — le clou du spectacle

【Dire】 « L'app a un HPA : au-delà de 70 % de CPU, elle passe de 3 à 10 réplicas automatiquement. Je génère de la charge. » **【Lancer】** Une fenêtre pour observer en direct (node1) :

```
watch -n2 kubectl get hpa,pods -n laravel-app-prod
```

【Lancer】 Une autre fenêtre pour charger (node1) :

```
kubectl -n ingress-nginx run load --rm -i --restart=Never --image=curlimages/curl -- \
  sh -c 'while true; do wget -q -O- http://laravel-app.laravel-app-prod.svc.cluster.local >/dev/nul
l; done'
```

【Voir】 Le CPU% du HPA monte → **REPLICAS** augmente ; la courbe CPU monte aussi dans Grafana. **Ctrl+C** pour arrêter la charge.

【Dire】 « Les règles d'anti-affinité répartissent les réplicas sur des nœuds différents : la panne d'un nœud ne coupe pas le service. » 【Lancer】

```
kubectl -n laravel-app-prod get pods -o wide # réplicas sur des NODE différents
```

3. GitOps (ArgoCD)

【Dire】 « Le déploiement est en GitOps : Git est la source de vérité, ArgoCD réconcilie en continu. » 【Lancer】

```
kubectl -n argocd get applications
```

【Voir】 **laravel-app** = **Synced / Healthy**. 【Dire】 (compléter) « Les composants d'infra en OutOfSync sont ceux qu'on a ajustés à la main pendant la mise au point ; l'app métier, elle, est synchronisée. » 【Lancer】 (navigateur, poste) ouvrir <http://argocd.35.156.165.130.nip.io>, montrer le graphe des applications.

4. Persistance + sauvegardes

【Dire】 « La base a un volume persistant ; la sauvegarde est un CronJob quotidien, déclenchable aussi à la demande. » 【Lancer】

```
kubectl -n laravel get pvc # volumes Bound
kubectl -n laravel create job --from=cronjob/mysql-backup demo-backup
sleep 8; kubectl -n laravel get pods | grep backup # la sauvegarde tourne
```

【Voir】 PVC **Bound** ; le job de backup apparaît puis passe **Completed**.

5. Sécurité — validating webhook (OPA / Gatekeeper)

【Dire】 « Contrôle d'admission : un pod non conforme est refusé. Celui-ci n'a pas le label obligatoire. » 【Lancer】

```
kubectl -n laravel-app-prod run bad --image=nginx --restart=Never
```

【Voir】 Erreur **denied the request ... label obligatoire manquant** (refusé).

【Dire】 « Avec le label, il passe. » 【Lancer】

```
kubectl -n laravel-app-prod run good --image=nginx --restart=Never --labels=app.kubernetes.io/name=demo
kubectl -n laravel-app-prod delete pod good
```

【Voir】 **pod/good created** (accepté).

6. Sécurité — isolation réseau (NetworkPolicy)

【Dire】 « Seuls l'ingress et le monitoring peuvent joindre l'app ; les autres namespaces sont bloqués. » 【Lancer】
Source autorisée (doit réussir) :

```
kubectl -n ingress-nginx run t --rm -i --restart=Never --image=curlimages/curl -- \
  curl -s -o /dev/null -w "ingress-nginx -> ${http_code}\n" http://laravel-app.laravel-app-prod.svc.c.cluster.local
```

【Lancer】 Source bloquée (doit expirer = 000) :

```
kubectl -n default run x --rm -i --restart=Never --image=curlimages/curl -- \
  curl -s -m6 -o /dev/null -w "default -> ${http_code}\n" http://laravel-app.laravel-app-prod.svc.c.luster.local
```

【Voir】 `ingress-nginx -> 200`, `default -> 000`.

7. Sécurité — secrets + HTTPS + SSO

【Dire】 « Les secrets sont dans des Secrets Kubernetes, pas en clair. » 【Lancer】

```
kubectl -n laravel-app-prod get secret
```

【Dire】 « cert-manager émet automatiquement un certificat TLS (HTTPS). » 【Lancer】

```
kubectl -n laravel-app-prod get certificate
kubectl get clusterissuer
```

【Voir】 Certificat `Ready=True`.

【Dire】 « Un SSO OIDC (dex + oauth2-proxy) protège les outils ; un accès non authentifié est renvoyé vers la page de login. » 【Lancer】

```
ING=$(kubectl -n ingress-nginx get svc ingress-nginx-controller -o jsonpath='{.spec.clusterIP}')
curl -s -o /dev/null -w "${http_code} -> ${redirect_url}\n" -H "Host: prometheus.kubequest.local" http://$ING/oauth2/start
```

【Voir】 `302 -> http://auth.kubequest.local/dex/auth?...` (redirection vers dex).

8. Monitoring (métriques)

【Dire】 « Prometheus collecte les métriques de tout le cluster, Grafana les visualise, 28 dashboards préconfigurés. »

【Lancer】 (navigateur, poste) ouvrir <http://grafana.35.156.165.130.nip.io> → ☰ → Dashboards → **Node Exporter / Nodes**. 【Voir】 Courbes CPU/RAM/réseau par nœud (encore mieux pendant la charge de l'étape 2).

8bis. Système de logs (Loki) — à montrer absolument

【Dire】 « Les logs sont centralisés avec Loki : un agent **Promtail** (DaemonSet) tourne sur chaque nœud et remonte les logs de tous les pods. Les **10 namespaces** du cluster y sont. »

【Lancer】 Prouver que le système tourne (node1) :

```
kubectl -n monitoring get pods | grep -E "loki|promtail" # loki-0 + un promtail par nœud
```

Méthode A (recommandée, visuelle) : Grafana → **Explore (icône boussole, à gauche)** → sélecteur de source de données → **Loki** → saisir :

```
{namespace="ingress-nginx"}
```

→ **Run query.** **【Voir】** Des lignes de logs horodatées — on voit en direct chaque requête HTTP faite depuis le navigateur/curl (GET/POST, URL, code, User-Agent). **【Dire】** « On peut filtrer par namespace/pod/label et faire de la recherche plein texte. » Par exemple les logs de l'app, ou seulement les erreurs :

```
{namespace="laravel-app-prod"}  
{namespace="laravel-app-prod"} |= "error"
```

Méthode B (ligne de commande, affiche les logs des 30 dernières minutes) (node1) :

```
kubectl -n monitoring run lokilog --rm -i --restart=Never --image=nicolaka/netshoot -- \
  sh -c 'E=$(date +%s)000000000; S=$(( $(date +%s) - 1800 ))000000000; curl -s -G http://loki.monit
oring:3100/loki/api/v1/query_range --data-urlencode "query={namespace=\"ingress-nginx\"}" --data-ur
lencode "limit=20" --data-urlencode "start=$S" --data-urlencode "end=$E" | jq -r ".data.result[].va
lues[][1]"'
```

【Voir】 Les vraies lignes de logs d'accès de l'ingress (dont le trafic de votre démo).

Remplacer **ingress-nginx** par **laravel-app-prod** / **argocd** / **monitoring**, etc. (10 namespaces disponibles).

9. Déploiement roulant + rollback automatique (exigence explicite de la défense)

【Dire】 « Je montre un déploiement qui échoue : le rolling update s'arrête, garde les anciens réplicas, l'app ne coupe pas, puis rollback en une commande. » **【Lancer】** Déployer une version cassée (probe qui échoue) :

```
kubectl -n laravel-app-prod set image deploy/laravel-app laravel=nginx:alpine      # mauvaise image vo
lontairement
kubectl -n laravel-app-prod rollout status deploy/laravel-app --timeout=40s      # va bloquer/expir
er
kubectl -n laravel-app-prod get pods                                           # nouveaux non prê
ts, anciens Running
```

【Dire】 « L'app n'a jamais coupé (les anciens réplicas servent), je rollback. » **【Lancer】**

```
kubectl -n laravel-app-prod rollout undo deploy/laravel-app
kubectl -n laravel-app-prod rollout status deploy/laravel-app --timeout=60s
```

【Voir】 Retour à la version saine, 2/2 Running.

⚠ Ce deploy est géré par ArgoCD (selfHeal) : après le rollback, ArgoCD le remet aussi à l'image déclarée dans Git. Vérifier avant la démo que l'image est **kubequest-app:secure2**.

10. Clôture : la phrase de conclusion

【Dire】 « En résumé : un cluster auto-construit à 4 nœuds à rôles séparés, un déploiement complet (Helm/Kustomize/ArgoCD), de la haute dispo (HPA/anti-affinité), persistance + sauvegardes, une observabilité complète (Prometheus/Grafana/Loki), et toute une couche de sécurité (Secrets, OPA, NetworkPolicy, HTTPS, SSO) avec un vrai audit de sécurité et un correctif de CVE. »

Secours (si ça déraile en démo)

Situation	Réflexe
Commande ne passe pas	<code>ssh node1</code> d'abord ; être sur le control-plane.
Un nœud NotReady	Sûrement éteint → le démarrer dans la console AWS.
Une page web ne charge pas	Utiliser les URL nip.io (pas les .local).
Un pod bloqué	<code>kubectl -n <ns> describe pod <nom></code> (voir Events).
OPA bloque une action légitime	Supprimer la contrainte : <code>kubectl delete k8srequiredlabels pods-must-have-name-label</code> .
Surtout NE PAS	Ne pas « Sync » <code>infra-security</code> dans ArgoCD (casse le cluster) ; pas de Sync global.